

fonctions

October 5, 2026

1 Les fonctions en Python

Ce document est disponible en version Jupyter Notebook. Pour le télécharger:

`git clone https://github.com/l1maht/Python-Aide`

Une fonction est un petit bloc de code auquel on donne un nom. Elle sert à effectuer une tâche que l'on peut réutiliser. On écrit la fonction une fois, puis on peut l'appeler plusieurs fois.

1.1 1. Définir et appeler une fonction

On définit une fonction avec le mot **def**, puis son nom et des parenthèses. Les instructions qui appartiennent à la fonction sont indentées. Pour la faire fonctionner, on l'appelle en écrivant son nom suivi de parenthèses.

```
[1]: def dire_bonjour():  
      print("Bonjour !")  
  
      dire_bonjour()
```

Bonjour !

1.2 2. Les paramètres : donner des informations à la fonction

Un **paramètre** est un nom écrit entre les parenthèses dans la définition. Il représente une information dont la fonction a besoin. Quand on appelle la fonction, la valeur qu'on lui donne s'appelle un **argument**.

Ici, **prenom** est le paramètre et 'Leina' est l'argument. On met un paramètre quand la fonction doit pouvoir travailler avec des valeurs qui peuvent changer d'un appel à l'autre. Si elle n'a besoin d'aucune information variable, les parenthèses peuvent rester vides.

Exécute et teste :

```
[2]: def saluer(prenom):  
      print('Bonjour', prenom, '!')  
  
      saluer('Leina')  
      saluer('Yanis')
```

Bonjour Leina !

Bonjour Yanis !

1.3 3. return : récupérer le résultat

`return` renvoie une valeur au code qui a appelé la fonction. On peut ensuite ranger cette valeur dans une variable, l'afficher ou la réutiliser dans un calcul. `print()` affiche du texte à l'écran, mais ne le renvoie pas : ce n'est pas la même chose.

Dans cet exemple, la fonction reçoit deux nombres et renvoie leur produit.

```
[3]: def multiplier(nombre_a, nombre_b):  
      return nombre_a * nombre_b  
  
resultat = multiplier(6, 7)  
print('Le produit est', resultat)
```

Le produit est 42

1.4 4. Variables extérieures et variables locales

Une variable créée **dans** une fonction est généralement **locale** : elle est utilisable dans cette fonction, mais pas directement à l'extérieur. Une variable créée en dehors d'une fonction est extérieure à cette fonction. La fonction peut lire une variable extérieure, mais si elle crée une variable du même nom, cette nouvelle variable est locale et ne remplace pas l'originale.

Garder les informations dans les paramètres et les variables locales rend les fonctions plus faciles à comprendre et à tester.

Exécute le code : le test confirme que la variable extérieure n'a pas changé.

```
[4]: message = 'Je suis en dehors de la fonction'  
  
def creer_message_local():  
    message = 'Je suis dans la fonction'  
    return message  
  
message_retourne = creer_message_local()  
print(message_retourne)  
print(message)  
  
# Tests : les deux variables du même nom sont distinctes.  
assert message_retourne == 'Je suis dans la fonction'  
assert message == 'Je suis en dehors de la fonction'
```

Je suis dans la fonction

Je suis en dehors de la fonction

1.5 5. Passer une liste à une fonction : le même objet

En Python, une liste passée en argument n'est pas automatiquement copiée. Le paramètre de la fonction désigne le **même objet liste** (ou **clé de la liste** si tu préfères) que la variable utilisée lors de l'appel. Donc, si la fonction modifie cette liste (par exemple avec `append`), on voit la modification depuis l'extérieur.

Attention : donner à un paramètre un nouveau contenu (`liste = [...]`) ne remplace pas la liste extérieure. Cela change seulement ce à quoi le nom local `liste` fait référence dans la fonction.

Dans le code suivant, le premier test vérifie la modification partagée; le second vérifie qu'une réaffectation locale ne remplace pas la liste d'origine.

```
[5]: def ajouter_fruit(liste, fruit):
    liste.append(fruit)

def remplacer_parametre_localement(liste):
    liste = ["poire"]
    return liste

fruits = ["pomme"]
ajouter_fruit(fruits, "banane")
print("Liste après append :", fruits)

nouvelle_liste_locale = remplacer_parametre_localement(fruits)
print("Liste retournée :", nouvelle_liste_locale)
print("Liste d'origine :", fruits)

# Tests : append modifie la liste partagée, mais la réaffectation ne la
↳ remplace pas.
assert fruits == ["pomme", "banane"]
assert nouvelle_liste_locale == ["poire"]
assert fruits == ["pomme", "banane"]
```

```
Liste après append : ['pomme', 'banane']
Liste retournée : ['poire']
Liste d'origine : ['pomme', 'banane']
```

1.6 6. La docstring : expliquer une fonction

Une **docstring** est une courte description placée juste après la ligne `def`, entre trois guillemets. Elle explique le rôle de la fonction et, si nécessaire, ses paramètres et ce qu'elle renvoie. Elle aide les personnes qui lisent le code; Python peut aussi l'afficher avec `help(nom_de_la_fonction)` ou `print(nom_de_la_fonction.__doc__)`.

```
[6]: def calculer_carre(nombre):
    """
    Renvoie le carré du nombre reçu en paramètre.

    :param nombre: Le nombre dont on veut calculer le carré.
    :type nombre: int

    :return: Le carré du nombre.
    :type: int
```

```
    """  
    return nombre * nombre  
  
print(calculer_carre(5))  
print(calculer_carre.__doc__)
```

25

Renvoie le carré du nombre reçu en paramètre.

:param nombre: Le nombre dont on veut calculer le carré.

:type nombre: int

:return: Le carré du nombre.

:type: int